Chapter 6: Returning Status

Until now, evaluation returned a value. A number node returned a number; addition returned a sum. A print statement performed its side effect and returned `None`. The caller could assume that evaluation had finished normally.

That assumption no longer holds when a computation asks to stop the program. We need to return both the value and the running program's **control-flow status**. Control flow describes how execution proceeds and where it transfers next. (1) It matters in interpreted and compiled programs alike. Our interpreter will represent that information explicitly as a Python pair:

```
value, status = evaluate(ast, environment)
```

The value carries the information we already returned. The status says whether normal computation can continue. We will introduce this contract using `exit` and `assert`, before adding conditionals or loops.

Breaking Changes

Earlier use	Change
A Python caller expects a value from `evaluate`	Unpack `(value, status)` instead.
A test expects print or assignment to return `None`	Expect `(None, None)`.
A variable is named `exit` or `assert`	Rename it; both names are now keywords.

The test helper `evaluate_source` now returns `((value, status), environment)`. Existing Vertex programs otherwise retain their behavior. Comparisons, explicit conversions, and `type()` still work. Logical operators still short-circuit. Print still requires parentheses, and semicolons still separate statements.

What the Pair Means

`None` in the status position means normal completion. A special status asks for a transfer of control. These are Python implementation values, not new Vertex values that source programs can inspect.

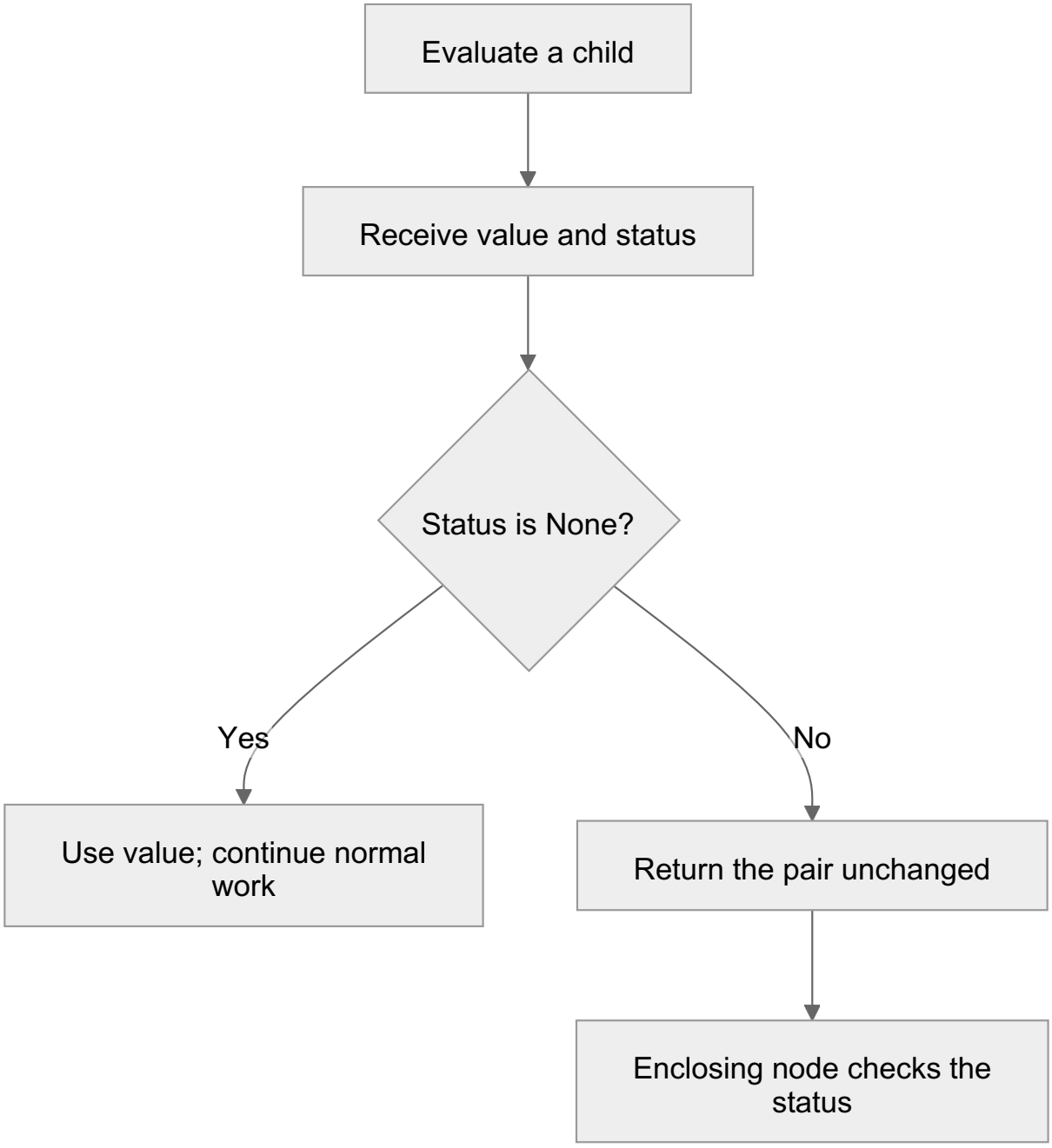
Evaluation	Returned pair	Meaning
`2 + 3`	`(5, None)`	Normal numeric result
`2 > 3`	`(False, None)`	Normal boolean result
`print(5)`	`(None, None)`	Normal completion after printing
`exit(7)`	`(7, "exit")`	Request termination with code 7
A failed assertion	`(1, "exit")`	Diagnostic already reported; request termination

A false value is not an abnormal status. Neither is a value of `None`. The two positions have different jobs, so every caller must check the status position rather than guessing from the value.

The status belongs to the evaluation result. We do not store a pending exit on the abstract syntax tree (AST), and we do not put it in the environment. The same tree can be evaluated again without carrying status from a previous run.

Check Before Continuing

Every parent follows the same rule: evaluate a child, inspect its status, and only then decide what to do with the value. If the status is special and this node does not handle it, return the pair unchanged. Do not evaluate another child or perform the parent's pending computation.



This diagram describes an ordinary node, not a handler for a special status. A structure that owns a particular request can handle it instead of propagating it. For now, the runner handles `"exit"`. Later, a loop can handle `"break"` without terminating the program.

For a binary operation, the rule applies twice. Here is the addition pattern, with type checks omitted so the status handling is easier to see:

```
left, status = evaluate(ast["left"], environment)
if status is not None:
    return left, status
right, status = evaluate(ast["right"], environment)
if status is not None:
    return right, status
return left + right, None
```

The first check prevents evaluation of the right operand after an exit on the left. The second prevents addition after an exit on the right. Type checks for the pending operation must also wait: an exit code is not an operand to add, convert, print, or test for truth.

Exit Starts the Transfer

Both forms are valid:

```
exit();
```

```
exit(2 + 3);
```

The first returns `(0, "exit")`. The second evaluates its argument and returns `(5, "exit")`. The argument is optional, but the parentheses are required. An explicitly supplied exit code must be an integer, not a float, boolean, or string.

Exit codes are how a completed process reports success or failure to its caller. Zero conventionally means success; nonzero codes indicate failure. (2) That numeric process code is distinct from our internal string status `"exit"`. Use small nonnegative codes in examples; operating systems can limit the range that the calling shell observes. (2)

`exit` is a statement, not a general expression: it can stand alone, but it cannot be written nested inside a larger expression, the way `1 + exit(7)` or `a[exit(7)]` might suggest. It is not a function call either. Nothing yet in Vertex can carry a status out from deep inside an expression the way a function body eventually will; a later chapter's function calls give a real way to exit from something nested, once a call exists to be nested in. For now, propagation is demonstrated at the level `exit` actually operates at: between statements, which the next section covers. In Extended Backus-Naur Form (EBNF):

```
exit_expression ::= "exit" "(" [ expression ] ")"
```

The brackets make the argument optional. The node records either that argument tree or `None`:

```
{"tag": "exit", "expression": {"tag": "number", "value": 7}}
{"tag": "exit", "expression": None}
```

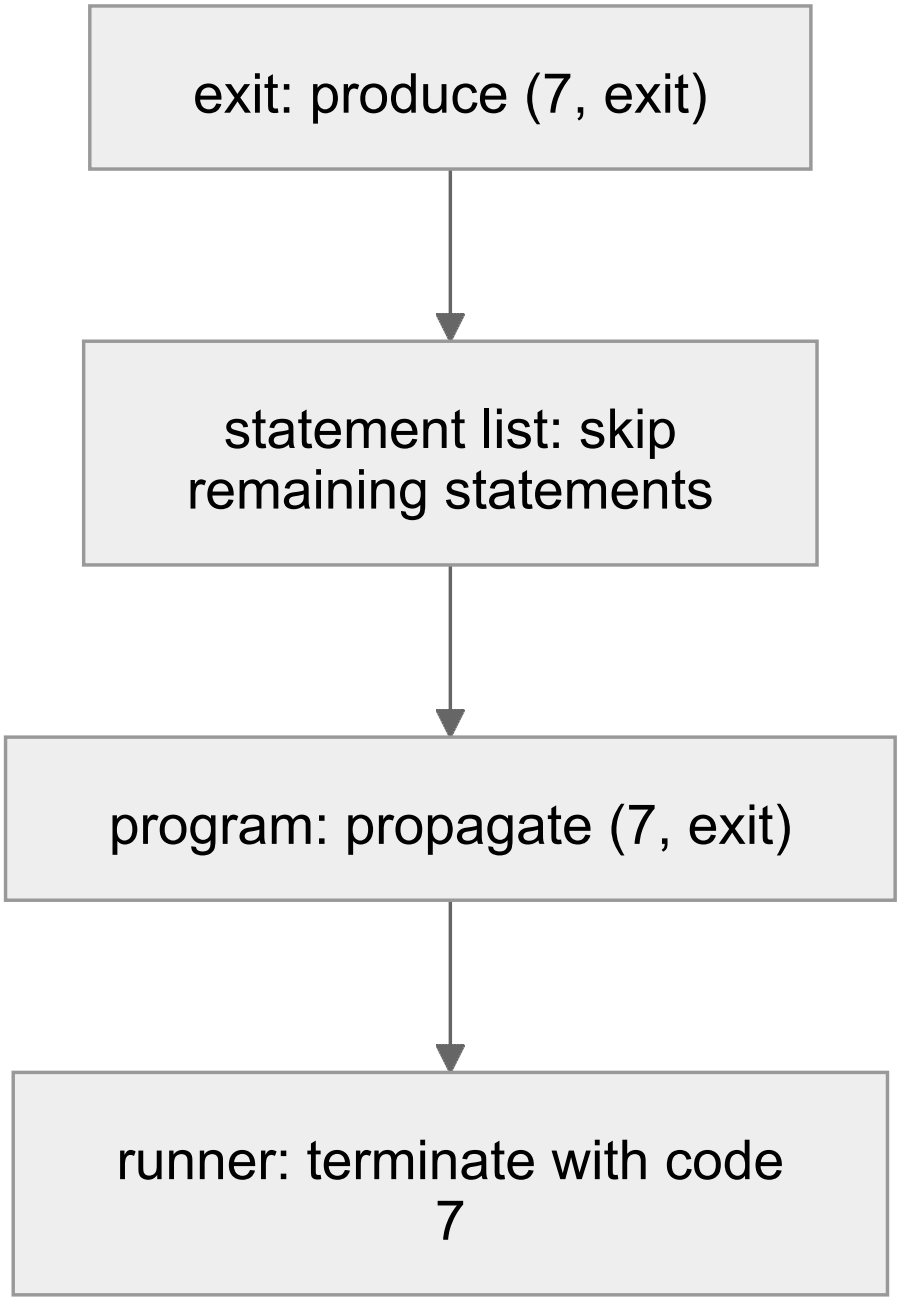
Following Exit Outward

Consider this complete program:

```
x = 3;
exit(7);
x = 99;
print(x);
```

The first assignment completes. The second statement is `exit(7)` on its own; evaluating it immediately produces `(7, "exit")`. Neither later statement runs: `x` is never reassigned to `99`, and the final print never executes.

The arrows below show evaluation results returning outward, not the initial order in which the evaluator visits nodes:



Every return between the exit node and the runner carries the same pair. This is a non-local transfer in the Vertex program: the request affects enclosing constructs, not just the statement that produced it. Our Python implementation achieves that transfer through ordinary function returns and explicit checks. No Python exception is needed for propagation through the evaluator.

The same discipline applies one level deeper too, inside a single expression, not just between statements – the next section's table and code excerpt show every node family checking a child's status before doing anything with its value. `exit` itself cannot currently sit inside one of those expressions to demonstrate that directly; nothing yet in this chapter can. The check-after- every-child rule is still there, waiting for something nested to exercise it.

Unwinding does not undo earlier effects. If an earlier statement had already read input before a later one requested exit, that read would remain consumed. Likewise, `x` remains `3` in this example. Status propagation prevents future work; it is not rollback.

Retrofitting the Existing Nodes

Changing only `exit` would not be enough. Every evaluator caller must honor the new result contract.

Node family	Work that must wait for normal status
Arithmetic and comparison	Evaluating the next operand and applying the operation
Unary operators, conversions, `type()`	Inspecting or transforming the operand value
Input with a prompt	Checking the prompt and consuming input
Assignment	Writing the binding
Print	Writing text and updating `__output`
Statement list	Executing the next statement

Literal and identifier evaluation still completes normally, but now returns the value paired with `None`. An undefined identifier still raises a Python error. This chapter does not turn every runtime error into a status.

The statement-list code makes the stopping rule explicit:

```
if ast["tag"] == "statement_list":
    for statement in ast["statements"]:
        value, status = evaluate(statement, environment)
        if status is not None:
            return value, status
    return None, None
```

Notice `is not None`, rather than a check specifically for `"exit"`. Ordinary nodes propagate special statuses without needing to know all their meanings. That leaves room for future `break`, `continue`, and `return` behavior.

Assertions Report, Then Exit

An assertion states a condition expected to hold at a particular point in a program. Checking it during execution can expose a violated assumption. (3) Vertex assertions are statements. They complete normally when their condition is true, but a false condition requests a non-local exit.

```
__input = "21";
answer = number(input()) * 2;
assert answer == 42, "Expected the supplied input to produce 42";
print(answer);
```

This prints `42`. Now consider a failing assertion:

```
answer = 41;
assert answer == 42, "Expected the supplied input to produce 42";
print("This must not run");
```

The assertion reports this diagnostic to standard error:

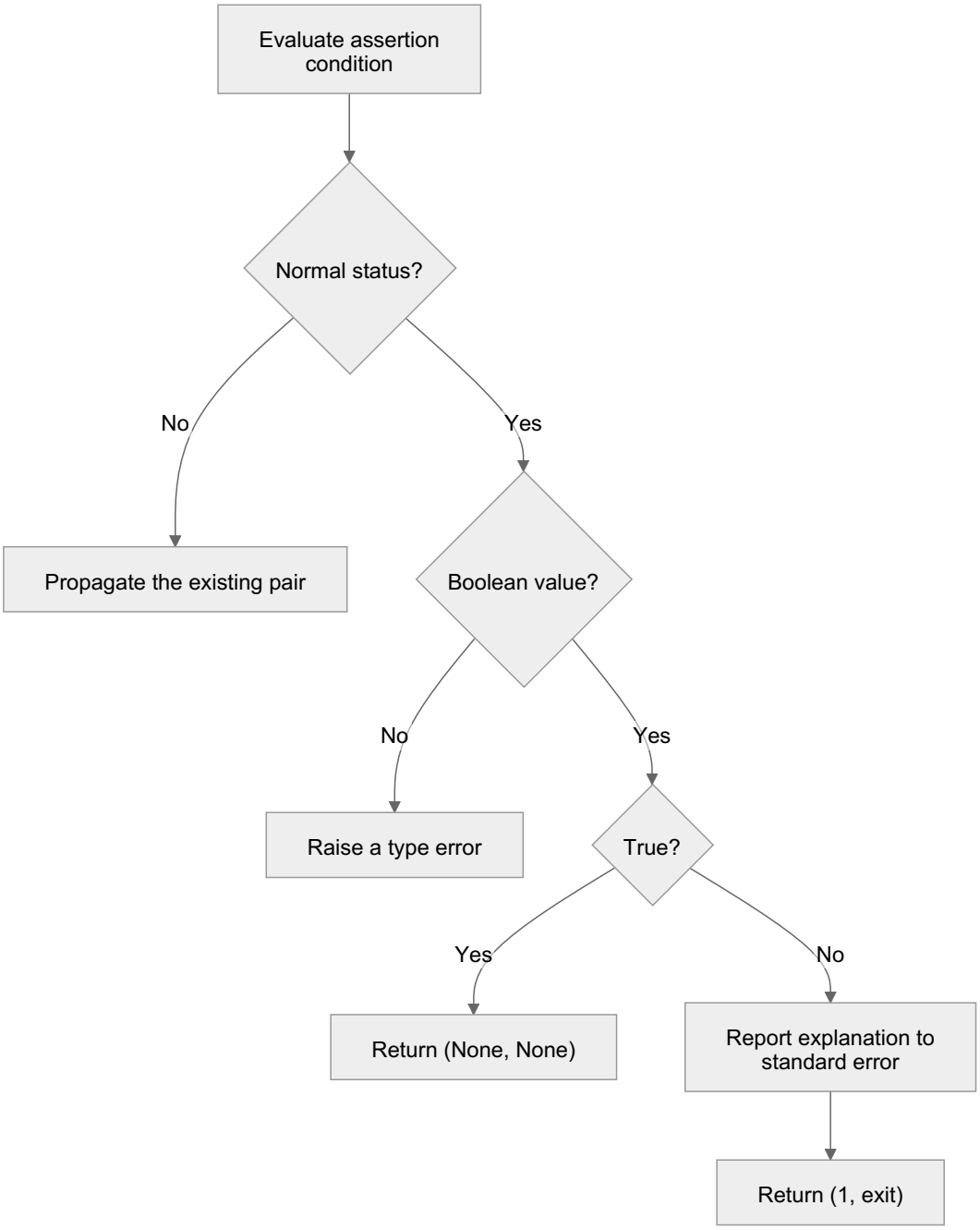
```
Assertion failed: Expected the supplied input to produce 42
```

It then returns `(1, "exit")`. The print is skipped, and the runner terminates with process code 1. The diagnostic does not travel through the AST. We report it where the failure is detected, then use the same status as `exit(1)`. There is no separate assertion status and no try/catch mechanism.

The explanation is mandatory. The expression tells us what was tested; the explanation states the expectation that failed. Write something useful about that expectation, rather than merely repeating “assertion failed.”

```
assert_statement ::= "assert" expression "," <string>
```

The explanation is a string literal, not an expression that runs on failure. Omitting the comma or string is a syntax error. The implementation accepts an empty string, but it provides no useful explanation. Assertions cannot be used as factors: `x = assert ...` is not valid syntax.



The first decision matters: when the condition's own evaluation already produced a status, `assert` passes it along unchanged, without ever asking whether the condition is a boolean or reporting a failure. `exit` cannot sit inside the condition's source text to show this directly, since it cannot nest inside any expression, but the evaluator does not care how a status arrived, only that one already did: given the condition node directly, as `{"tag": "exit", "expression": {"tag": "number", "value": 7}}`, `assert` propagates `(7, "exit")` without reporting anything. Conversely, `assert 1, "Expected true";` raises a type error: numbers do not implicitly become booleans.

The failure branch in the evaluator is short:

```
message = "Assertion failed: " + ast["explanation"]
# Report here, then request ordinary program exit. No exception payload
# needs to travel through the tree because Vertex has no catch mechanism.
print(message, file=sys.stderr)
return 1, "exit"
```

This is not Python's `assert` statement. Vertex performs its own check and its own reporting. The diagnostic also does not change `__output`, which records only the latest Vertex print statement.

The Runner Owns Process Termination

The program node passes a special status outward. The runner is where an exit request becomes process termination:

```
value, status = evaluate(ast, {})
if status == "exit":
    # Keep process termination outside the recursive evaluator.
    sys.exit(value)
if status is not None:
    raise ValueError(f"Unhandled evaluation status: {status}")
```

The runner does not print an assertion message again. Both an explicit exit and a failed assertion arrive as `"exit"`, carrying an integer code.

Keeping this boundary outside the evaluator lets tests evaluate an exiting program and inspect its returned pair without terminating the test process. Python's `sys.exit` requests termination by raising `SystemExit`; when uncaught, it terminates the process. (4) That host mechanism is used only at the boundary, not to propagate Vertex control flow through each node.

Short-Circuiting Still Applies

Status propagation only occurs for nodes that run. `exit` cannot sit inside a larger expression to prove that directly, but division by zero can stand in for it just as well: it raises immediately if it runs, and stays silent if it does not.

```
assert true or (1 / 0), "Left operand should decide the result";
saved = false and (1 / 0);
print(saved);
```

This prints `false` and completes normally. Neither division executes: if one did, it would raise a Python error immediately rather than return quietly. Short-circuit evaluation already skips an unnecessary operand, and the skip happens before that operand is evaluated at all, not after.

Expression	Result
`false and (1 / 0)`	`(False, None)`; the division never runs
`true or (1 / 0)`	`(True, None)`; the division never runs
`true and (1 / 0)`	`ZeroDivisionError`; the division does run
`false or (1 / 0)`	`ZeroDivisionError`; the division does run

A status, when one does occur, still must be checked before the operator uses its operand's value – that part of the rule does not depend on anything exit-specific; the "Retrofitting the Existing Nodes" table above already covers every node family that honors it, conversions and input's prompt included.

All source must still tokenize and parse successfully before evaluation begins. An earlier exit cannot hide a syntax error later in the program.

Testing the Contract

From `chapter-06-returning-status/code`, run:

```
bash run-tests.sh
./vertex example.v < /dev/null
```

The example remains noninteractive. It retains the earlier comparisons and conversions, checks assertions, skips a division by zero that would raise through logical short-circuiting, then exits as its own statement. The script checks its complete output.

Tests must inspect status as well as value. A numeric value of 7 could be a normal result or the payload of an exit. This test uses the current helper:

```
from evaluator import evaluate_source

result, environment = evaluate_source("x=3;exit(7);x=99")
assert result == (7, "exit")
assert environment["x"] == 3
```

The tests also capture standard error to verify that an assertion diagnostic appears exactly once. Mock input checks that skipped work does not read from the keyboard. Separate subprocess tests check that the runner produces the expected process code.

Conditional Statements will use this contract for conditions and selected branches. Loops will eventually handle their own `break` and `continue` requests, while passing program exits outward. Those constructs are not implemented in this version.

Considerations

1. Why is a false expression value different from an abnormal control-flow status?
2. What would go wrong if assignment stored the returned value before checking status?
3. Why should a loop handle `break` but allow `exit` to keep propagating?
4. What would need to change if a future catch construct could recover from an assertion failure? Would immediate reporting still be the right choice?

Practice

Answers to the practice questions follow this section.

1. Compare `exit(3);print("done");` with `print("done");exit(3);`. Check both output and status.
2. Write a failing assertion with an explanation of the expected value. Verify that a later assignment does not happen and `__output` does not change.
3. Supply input through `__input`, then compare `result=input()+(1/0)` with `result=(1/0)+input()`. Inspect whether the supplied input was consumed.
4. Find every recursive evaluator call in `evaluator.py`. Identify the status check that must occur before the caller continues.

Answers to the Practice Questions

1. `exit(3);print("done");` returns `(3, "exit")` without printing and leaves `__output` unchanged; the print statement never runs. `print("done");exit(3);` prints `done`, stores `"done"` in `__output`, then returns `(3, "exit")` too – the exit code is the same either way, since it comes from the literal `3`, not from anything print did; only the output differs, because order decides whether print gets to run before exit ends the program.

2. A failing assertion stops later work without replacing the saved output:

```
import io
from contextlib import redirect_stderr
from evaluator import evaluate_source

error = io.StringIO()
with redirect_stderr(error):
    result, environment = evaluate_source(
        'assert 2+2==5,"Expected five";later=1', {"__output": "saved"})
assert result == (1, "exit")
assert "later" not in environment
assert environment["__output"] == "saved"
assert error.getvalue() == "Assertion failed: Expected five\n"
```

3. Both expressions raise `ZeroDivisionError`, since addition does not short-circuit and both operands are always evaluated, left to right. `result=input()+(1/0)` consumes the supplied input first, then crashes evaluating the right operand; `__input` is already cleared by the time the error is raised. `result=(1/0)+input()` crashes evaluating its left operand before `input()` ever runs, leaving the supplied input waiting, exactly as `exit` would if it could still sit there to prove it.

4. After evaluating a child, the caller must inspect its status before performing the operation that needed its value. Trace the statement list, assignment and print, unary and binary operations, conversions, input prompts, assertions, and logical operations in this version. Logical operations still skip their right operand when short-circuiting; a skipped expression has no status to propagate. The statement list stops before the next statement when a child returns a status.

References

1. Wikipedia contributors. “Control flow.” Wikipedia. https://en.wikipedia.org/wiki/Control_flow
2. Wikipedia contributors. “Exit status.” Wikipedia. https://en.wikipedia.org/wiki/Exit_status
3. Wikipedia contributors. “Assertion (software development).” Wikipedia. https://en.wikipedia.org/wiki/Assertion_(software_development)
4. Python Software Foundation. “sys.exit.” Python documentation. https://docs.python.org/3/library/sys.html#sys.exit